Finite Automata in Haskell

V. Velkey, W. Vromen

Wednesday 24$^{\text{th}}$ May, 2023

Abstract

We implement finite state automata, both deterministic and nondeterministic. We also implement regular expressions and generate words from their language. Finally, we implement transitions between our objects and run several tests to verify our implementation.

Contents

1 Introduction

Finite state automata are mathematical models of computation. They are mostly used to study regular languages, i.e. languages with a certain structure. This structure is also captured by regular expressions.

In this report we discuss our Haskell-implementation of finite state automata and regular expressions. In particular, we implemented deterministic finite state automata, non-deterministic finite state automata and regular expressions. The goal is to be able to compare the languages generated by regular expressions and the languages accepted by the various automata.

In the first section we will provide some formal background for automata theory and regular languages. For the second part we will discuss our implementation. Afterwards in section three we will show some cool stuff / provable stuff that one can do with the program, after which there will be an epic conclusion in the final section.

2 DFA Implementation

This describes our implementation of Deterministic Finite State Automata. As in the definition, a DFA consists of a tuple $(Q, \Sigma, \delta, q_{start}, A)$ representing the set of states, the alphabet, the transition function, the start state and the accept states.

```
module DFA where

type State = Int
type Symbol = Char
data DFA = DFA { states :: [[State]] -- For ease with subset construct, represent by set of
     states.
                , alphabet:: [Symbol]
                , delta :: Symbol -> [State] -> [State]
                ,  start:: [State]
                , acceptstate:: [[State]]}
```

We implement a basic evaluation function that upon input from the string, evaluates if the string is in the language.

```
evaluate:: DFA -> String -> Bool
evaluate df st = all (`elem` alphabet df) st && -- captures that all element of string in
    alphabet
                stateArr (start df) st `elem` acceptstate df where
                    stateArr:: [State] -> String -> [State] -- recursively go to state at
                        end of string
                    stateArr q [] = q
                    stateArr q (x:xs) = stateArr (delta df x q) xs
```

We write a toy example DFA on the alphabet $\Sigma = \{0, 1\}$ computing the language of words starting with a 0.

```
zeroStart:: DFA
zeroStart = DFA [[0],[1],[2]] ['0', '1'] deltazero [0] [[1]] where
    deltazero:: Symbol -> [State] -> [State]
    deltazero char st
     | st == [0] && char == '0' = [1]
     | st == [0] && char == '1' = [2]
     | st == [1] = [1]
```

```
    | st == [2] = [2]
    | otherwise = [-1]
```

3 NFA Implementation

This describes our implementation of Nondeterministic Finite State Automata. As in the definition, an NFA consists of a tuple $(Q, \Sigma, \delta, q_{start}, A)$ representing the set of states, the alphabet, the transition function, the start state and the accept states.

```
module NFA where

import Data.List
import Data.Maybe()
import DFA
data NFA = NFA { statesNF :: [State]
                , alphabetNF:: [Symbol]
                , deltaNF :: Symbol -> State -> [State]
                , startNF:: State
                , acceptstateNF:: [State]}
```

We also implement a similar, but more complicated transition function suitable for NFA-s.

```
evaluateNF:: NFA -> String -> Bool
evaluateNF nf st = all (`elem` alphabetNF nf) st && -- captures elements of string in
    alphabet
          any (`elem` acceptstateNF nf) (stateArrNF' [startNF nf] st)  where
                stateArrNF':: [State] -> String -> [State]
                stateArrNF' qs [] = qs
                stateArrNF' [] _ = []
                stateArrNF' [q] (x:xs) | q == -1 = []
                                       | otherwise = stateArrNF' (deltaNF nf x q) xs --
                                         recursion on string
                stateArrNF' (q:qs) (x:xs) = stateArrNF' [q] (x : xs) `union` stateArrNF' qs
                     (x : xs) --recursion on statespace
```

4 Implementation of ϵ-NFA-s

Here we modify our implementation of NFA-s to account for epsilon transitions. This requires some technical apparatus. First we give the type structure and define some helper functions to calculate ϵ-closure of states. For this we use the blog answer [Jel14].

```
module ENFA where

import Data.List
import Data.Maybe()
import DFA
data ENFA = ENFA { statesENF :: [State]
                , alphabetENF:: [Symbol]
                , deltaENF :: Symbol -> State -> [State]
                , epTrans:: [(State, State)]
                , startENF:: State
                , acceptstateENF:: [State]}

-- Taken from https://stackoverflow.com/questions/19212558/transitive-closure-from-a-list-
    using-haskell
trClose :: Eq a => [(a, a)] -> [(a, a)]
trClose closure
  | closure == closureUntilNow = closure
```

```
  | otherwise                    = trClose closureUntilNow
  where closureUntilNow =
          nub $ closure ++ [(a, c) | (a, b) <- closure, (b', c) <- closure, b == b']


refClose:: Eq a => [a] -> [(a,a)] -> [(a,a)]
refClose as ps = nub (ps ++ [(x,x) | x <- as])

rtClose:: ENFA -> State -> [State]
rtClose nf q = [p | p <- statesENF nf , (q, p) `elem` trClose rcl ] where
                                              rcl = refClose (statesENF nf) (epTrans nf)
```

Finally, we modify the the evaluation function.

```
unionL:: Eq a => [[a]] -> [a]
unionL = foldr union []

evaluateENF:: ENFA -> String -> Bool
evaluateENF nf st = all (`elem` alphabetENF nf) st && -- captures elements of string in
    alphabet
             any (`elem` acceptstateENF nf) (stateArrENF' [startENF nf] st)  where
                stateArrENF':: [State] -> String -> [State]
                stateArrENF' qs [] = qs
                stateArrENF' [] _ = []
                stateArrENF' [q] (x:xs) | q == -1 = []
                                        | otherwise = stateArrENF' (unionL (map (deltaENF
                                            nf x) (rtClose nf q))) xs
                stateArrENF' (q:qs) (x:xs) = stateArrENF' [q] (x : xs) `union` stateArrENF'
                     qs (x : xs) --recursion on statespace
```

5 Regular expressions

Here we implement regular expressions and a toolset to generate words that the regexp accepts.

```
module RegExp where

import System.Random
import DFA
import NFA()
import ENFA()

data RegExp = Epsilon | R [Symbol] | Union RegExp RegExp | Star RegExp | Con RegExp RegExp
    | Plus RegExp

instance Show RegExp where
  show Epsilon = "R e"
  show (R xs) = "R " ++ show xs
  show (Union r1 r2) = show r1 ++ " u " ++ show r2
  show (Star r) = "(" ++  show r ++ ")*"
  show (Con r1 r2) = show r1 ++ show r2
  show (Plus r) = "(" ++ show r ++ ")+"


generateWord :: RegExp -> IO String
generateWord Epsilon = pure ""
generateWord (R xs) = pure xs
generateWord (Union r1 r2) = do
  i <- getStdRandom (randomR (0,1::Int))
  [generateWord r1, generateWord r2] !! i
generateWord (Con r1 r2) = do
  one <- generateWord r1
  two <- generateWord r2
  return (one ++ two)
generateWord (Star r) = do
  i <- getStdRandom (randomR (0,1::Int))
  [generateWord Epsilon, generateWord (Con r (Star r))] !! i
```

```
generateWord (Plus r) = generateWord (Con r (Star r))
```

A function to use for testing:

```
qc :: DFA -> RegExp -> Int -> IO ()
qc _ _ 0 = print "No counterexample found"
qc dfa ex n =  do
  s <- generateWord ex
  if evaluate dfa s
    then qc dfa ex (n-1)
    else
      if s == ""
      then print "The DFA rejected the empty string"
      else print ("the DFA rejected " ++ s)
```

6 Transition functions

Here we implement several transition functions between objects. First the powerset construction between NFA-s and DFA-s.

```
module Transitions where
import Data.List ( intersect, subsequences )
import NFA
import DFA
import ENFA



transNtoD:: NFA -> DFA
transNtoD (NFA sts alph del strt ac) =
  DFA (subsequences sts) alph del' [strt] [st | st <- subsequences sts,  intersect st ac /=
       []] where
    del':: Symbol -> [State] -> [State]
    del' sy ls = unionL [del sy l | l <- ls]
```

We extend the powerset construction for ϵ-NFA-s.

```
transENtoD:: ENFA -> DFA
transENtoD (ENFA sts alph del eps strt ac) = let nf = ENFA sts alph del eps strt ac in
  DFA (subsequences sts) alph del' (rtClose nf strt) [st | st <- subsequences sts,
      intersect st ac /= []] where
    del':: Symbol -> [State] -> [State]
    del' sy ls = let nf = ENFA sts alph del eps strt ac in  unionL (map (rtClose nf) lis)
        where
                                                              lis = unionL [del sy l | l <-
                                                                   ls]
```

7 Simple Tests

We now use the library QuickCheck to randomly generate input for our functions and test some properties.

We will later add stuff to (pseudo)test whether some automata are equal, whether some regular expression generates exactly the language of some automaton, and we will add tests that show that our definitions work.

```
module Main where

--import Test.QuickCheck
```

Note that the first test is a specific test with fixed inputs. The second and third test use QuickCheck.

```
main :: IO ()
main = print ":)"
```

8 Conclusion

Our primary sources of reference are [Chi18] and [Sip13].

References

[Chi18] Maurice Chiodo. Automata and formal languages lecture notes, December 2018.

[Jel14] Tikhon Jelvis. Transitive closure from a list using haskell. Computer Science Stack Exchange, 2014. [Online:] `https://stackoverflow.com/questions/19212558`.

[Sip13] Michael Sipser. *Introduction to the Theory of Computation*. Course Technology, Boston, MA, third edition, 2013.